

Part 5 — Force Calculations

Physics, formulas, MATLAB architecture, expected values

Your geometry engine is verified. This is everything downstream of it.

Every expected value below was computed from your actual hardpoint file and `generalinputs.m`. If your code produces these numbers, it's right.

PART A — Architecture

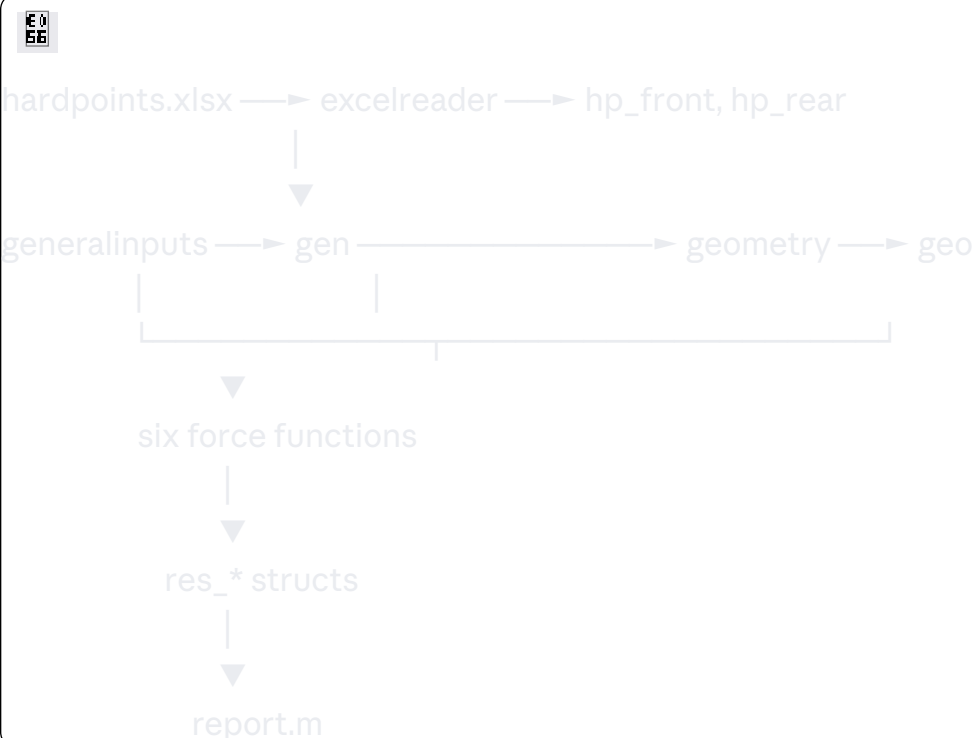
A.1 Function signatures

Every force function takes the same two structs and returns one:

```
function res = frontknuckle(gen, geo)
function res = rearknuckle(gen, geo)
function res = aArms(gen, geo)
function res = rockers(gen, geo)
function res = hubs(gen, geo)
function res = rackpinion(gen, geo)
```

Uniform signature means you can call them in a loop later, and you never have to remember which one needs what.

A.2 Data flow



A.3 main.m

Six lines of substance:

`clear; clc; close all` and `format long g`

Run `generalinputs` (it's a script, so it drops `gen` into the workspace)

Call `excelreader` with the filename and `gen.loaded_radius`

Call `geometry`

Call the six force functions, storing each result

Call `report`
Wrap steps 5–6 in a `try/catch` if you want, but with three hours left, don't bother — you want errors loud.

A.4 THE UNIT RULE — read this before writing anything

Your geometry is in **millimetres**. Your `gen` accelerations and speeds are in **SI** (m/s, m/s²).

Multiply those together and you get nonsense — silently, with no error.

Rule: at the top of every force function, pull the geometry values you need and divide by 1000.

Do it once, in a clearly-marked block:

```
EO
EE

% ---- unpack geometry, mm -> m ----
= geo.wheelbase / 1000;
Tf = geo.trackwidth_front / 1000;
a = geo.upphubsep_front / 1000;
..
```

Then every formula below is pure SI: forces in **N**, torques in **N·m**. That matches Report 9.0's published units directly, so comparison is trivial.

The one exception: spring rates stay in N/mm and bump travel in mm, because `N/mm × mm = N` is already consistent. Don't convert those.

A.5 Two `gen` values need changing

Field	Current	Change to	Why
<code>gen.bump</code>	30	25.4	Report 9.0's bump travel; needed for comparable rocker/hub numbers
<code>gen.c</code>	comment says <code>% m</code>	<code>% mm</code>	Value is already 55 mm, comment is stale

And two need adding:

Field	Value	Source
<code>gen.MR_F</code>	1.429	LOTUS bump sweep, revised model
<code>gen.MR_R</code>	1.592	LOTUS bump sweep

Motion ratio is the one quantity your static hardpoint file genuinely can't give you — it needs a full kinematic sweep, which LOTUS already did. This is a legitimate hardcode.

PART B — Front knuckle

B.1 What the part does

The **upright** (knuckle) is the structural bridge between the wheel and the suspension. The wheel bearing sits in it; both wishbones attach via ball joints; the steering arm takes the tie rod; the brake caliper bolts to it.

Every force the tyre generates reaches the chassis through this part. It's the highest-consequence component in the corner.

B.2 Braking case — physics

When you brake, three things happen at once:

Weight shifts forward. The car's inertia acts at CG height while the ground reaction acts at ground level.

That couple pushes load onto the front axle.

Friction acts at the contact patch, limited by $\mu \times F_z$.

The caliper reacts brake torque into the upright.

The upright is then a short beam held at two points (the ball joints), loaded by the friction force coming through the bearing and by the caliper couple.

B.3 Symbols

Symbol	Meaning	Unit
F_w	Static vertical load on one front wheel	N
ΔF	Longitudinal load transfer onto one front wheel	N
F_v	Total vertical load on that wheel while braking	N
f	Friction (braking) force at the contact patch	N
T_b	Brake torque about the wheel axis	N·m
a	Vertical distance, wheel centre up to upper ball joint	m
b	Vertical distance, wheel centre down to lower ball joint	m
r	Loaded tyre radius	m
F_1	Reaction at the upper ball joint	N
F_2	Reaction at the lower ball joint	N

B.4 Formulas


$F_w = M_f \cdot g / 2$
$\Delta F = M \cdot a_{\text{brake}} \cdot h / (2 \cdot L)$
$F_v = F_w + \Delta F$
$f = \mu \cdot F_v$
$T_b = f \cdot r$
$F_1 = (f \cdot b - T_b) / (a + b)$
$F_2 = f - F_1$

B.4a Derivation of F1 and F2

Step 1 — choose the free body

Cut the car so that the **upright alone** is isolated. Everything that crosses the cut becomes an external force on it.

This is the choice that makes brake torque *visible*. If you instead isolated the wheel-plus-upright, the caliper would be pushing on the rotor and the rotor pushing back on the caliper — both inside the boundary, so the brake torque would cancel and never appear. Cutting at the bearing forces it into the open.

Step 2 — what crosses the cut

Four things:

Source	What it transmits	Where it acts
Wheel bearing	Force f rearward	Wheel centre
Brake caliper	Couple T_b	Anywhere — a couple has no point
Upper ball joint	Force F_1 , longitudinal	Distance a above wheel centre
Lower ball joint	Force F_2 , longitudinal	Distance b below wheel centre

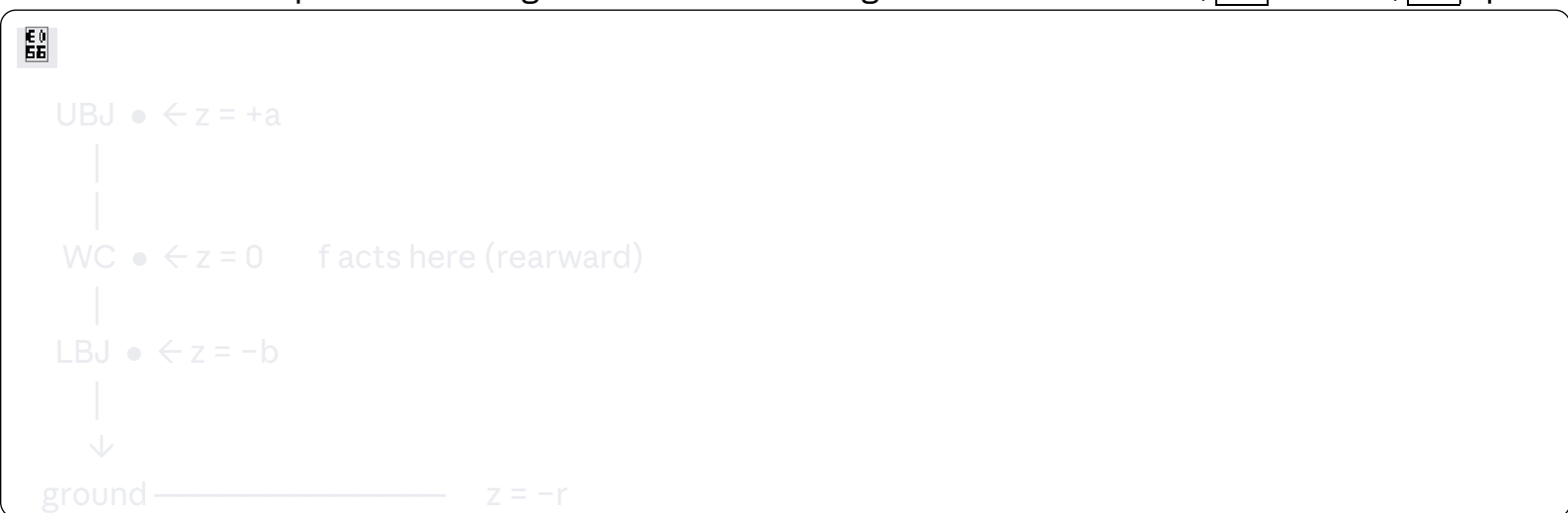
Two notes on why the list is this short:

The bearing transmits no moment about the spin axis. That's its whole job — it lets the hub rotate freely. So the only thing that can hand a spin-axis moment to the upright is the caliper, which is bolted to it.

The ball joints transmit no moment either. They're spherical bearings. That's what makes the wishbones two-force members and the upright a determinate three-force problem.

Step 3 — set up the geometry

Work in the vertical plane containing the wheel. Put the origin at the wheel centre, $+x$ forward, $+z$ up.



Measured from the **lower ball joint**, the upper ball joint is $(a + b)$ above it and the wheel centre is b above it.

Step 4 — moment equation

Take moments about the lower ball joint. That kills F_2 — it has zero moment arm about its own point — leaving one equation in one unknown.

Three contributions:

Term	Magnitude	Sense
Bearing force f at arm b	$f \cdot b$	tips the upright one way
Caliper couple	T_b	tips it the other
Upper ball joint F_1 at arm $(a+b)$	$F_1 \cdot (a + b)$	reacts the net

$$\sum M_{LBJ} = 0 \implies F_1(a + b) + T_b - f b = 0$$

$$F_1 = \frac{f b - T_b}{a + b}$$

Step 5 — force equation

Horizontal equilibrium. The bearing pushes f rearward; the two ball joints together must push f forward:

$$\sum F_x = 0 \implies F_1 + F_2 = f \implies F_2 = f - F_1$$

Step 6 — read the signs

With your numbers: $f = 614.33 \text{ N}$, $b = 0.10284 \text{ m}$, $T_b = 144.25 \text{ N}\cdot\text{m}$, $a+b = 0.17980 \text{ m}$

$$F_1 = \frac{614.33 \times 0.10284 - 144.25}{0.17980} = \frac{63.18 - 144.25}{0.17980} = -450.87 \text{ N}$$

$$F_2 = 614.33 - (-450.87) = +1065.21 \text{ N}$$

F_1 is negative, and that's physically correct. The two ball joints form a **couple**: the upper pulls rearward while the lower pushes forward. They have to, because the caliper is trying to rotate the upright about the wheel axis and only a couple can resist a couple.

Why the brake term wins. Compare the two competing quantities:

$$T_b = f \cdot r = f \times 234.80 \quad \text{vs} \quad f \cdot b = f \times 102.84$$

The brake torque acts through the **tyre radius**; the bearing force acts through the much shorter **ball joint offset**. Since $r > b$ on any real car, T_b always dominates and F_1 is always negative during braking. Report 9.0 got a negative F_1 too, for the same reason.

Step 7 — independent check

Take a completely different free body: **wheel + hub + upright together**. Now the brake torque is internal and vanishes, and the only external horizontal force is friction acting at the contact patch, $(r - b)$ below the lower ball joint.

$$F_1(a + b) + f(r - b) = 0 \implies F_1 = -\frac{f(r - b)}{a + b}$$

Substitute your numbers:

$$F_1 = -\frac{614.33 \times (0.23480 - 0.10284)}{0.17980} = -450.87 \text{ N}$$

Identical. And algebraically it must be — expand $T_b = f \cdot r$ in the Step 4 result:

$$\frac{f b - f r}{a + b} = - \frac{f(r - b)}{a + b}$$

Two different cuts through the car, two different equations, one answer. That's the check worth running when a free-body result looks wrong: pick a different boundary and see whether you land in the same place.

B.5 Cornering case — physics

Cornering loads the outer wheel through two mechanisms:

Centrifugal couple. The lateral inertia force acts at CG height; the ground reaction is at ground level. That couple transfers vertical load from inner to outer wheels.

Gyroscopic effect. A spinning wheel resists having its axis tilted. Yawing the car precesses each wheel, producing a small extra vertical load. It's genuinely small — about 5% of the centrifugal term — but Report 9.0 includes it, so keep it.

B.6 Cornering symbols and formulas

Symbol	Meaning
F_c	Vertical load from the centrifugal couple
F_g	Vertical load from the gyroscopic effect
F_{net}	Total vertical load on the outer front wheel
F_L	Lateral force at the contact patch
F_3 , F_4	Ball joint reactions to the lateral force

$$F_c = M \cdot v^2 \cdot h / (2 \cdot R \cdot T_f)$$

$$F_g = 4 \cdot m_{wheel} \cdot v^2 \cdot r / (2 \cdot R)$$

$$F_{net} = F_w + F_c + F_g$$

$$F_L = \mu \cdot F_{net}$$

$$F_3 = F_L \cdot (r - b) / (a + b)$$

$$F_4 = F_L + F_2$$

On F_g : Report 9.0 writes it as $4 \cdot m \cdot v^2 \cdot r^2 / (r \cdot 2 \cdot R)$, which simplifies to the form above — one r cancels. Code the simplified version; it's the same number and harder to mistype.

B.7 Steering arm force

Symbol	Meaning
<code>fo</code>	Friction force used for the steering calculation
<code>rs</code>	Scrub radius — lever arm from the steering axis to the contact patch
<code>Ts</code>	Torque about the steering axis
<code>Ls</code>	Effective steering arm length
<code>F5</code>	Tie rod force

```

E0
E6

fo = μ · Fw
Ts = fo · rs
F5 = Ts / Ls

```

The physics: braking force at the contact patch sits `rs` sideways from the steering axis, so it tries to twist the wheel about that axis. The tie rod resists at lever `Ls`.

B.8 MATLAB guidance

Structure it in three `%%` sections — braking, cornering, steering. Run each with `Ctrl+Enter` and check against the table before writing the next.

Unpack first. Pull `a`, `b`, `r`, `L`, `Tf`, `rs`, `Ls` into short local names with the `/1000` applied. Every formula then reads like the maths.

One named variable per line, matching the formula order. Don't inline — you want every intermediate visible when a number comes out wrong.

Store into `res` with descriptive fields: `res.Fw`, `res.Tb`, `res.F1_upperBJ`, `res.F5_tierod`. Your ANSYS setup reads these names.

Which geometry field is `a` and which is `b`: `geo.upphubsep_front` is `a` (76.96 mm), `geo.lowhubsep_front` is `b` (102.84 mm).

B.9 Expected values

Quantity Value Report 9.0

Fw	559.17 N	559.2
ΔF	294.07 N	294.25
Fv	853.24 N	853.45
f	614.33 N	614.48
Tb	144.25 N·m	140.47
F1	−450.87 N	−326.09
F2	1065.21 N	940.59
Fc	1763.24 N	1763.2 ✓
Fg	122.29 N	119.06
Fnet	2444.70 N	2441.5
FL	1760.18 N	1757.9
F3	1291.85 N	932.82
F4	2825.39 N	2698.4
fo	402.60 N	402.6 ✓
Ts	25.681 N·m	25.67 ✓
F5	570.03 N	570.53

Where you diverge and why:

Tb

 differs 2.7% because your loaded radius is 234.80 mm, not 228.6

F1

,

F2

,

F3

,

F4

 differ 15–38% because your derived

a

 and

b

 are **76.96 / 102.84**, not the symmetric 110.9 / 110.9 in Report 9.0

Both differences are *your numbers being more correct*. Report them as such — a comparison column in

report.m

 turns this into a finding rather than a discrepancy.

PART C — Rear knuckle

C.1 What differs from the front

No steering — a fixed toe link instead of a tie rod. And it takes drive torque as well as brake torque.

C.2 Formulas

$W_r = M_r \cdot g / 2$

$F_v = W_r + \Delta F$ (accelerating: rear gains load)

$F_{z_outer} = W_r + M \cdot a_{corner} \cdot h / (2 \cdot Tr)$

$F_c = \mu \cdot F_{z_outer}$ cornering force

$F_b = \mu \cdot F_v$ braking force

$T_b = F_b \cdot c$ c = brake mount to upright centre

$F_t = \mu \cdot F_v$ toe link force

Report 9.0 uses $F_c = 3 \cdot F_v$ and $F_b = 1.5 \cdot W_r$. Those are unexplained multipliers with no stated basis. Deriving from μ is defensible and gives you something you can justify in Design Event. Note the change in your report.

C.3 Expected values

Quantity	Value	Report 9.0
----------	-------	------------

W_r	667.08 N	667.1 ✓
-------	-----------------	---------

F_v	961.15 N	961.38 ✓
-------	-----------------	----------

F_c	829.52 N	2884.1 (their $3 \cdot F_v$)
-------	-----------------	-------------------------------

F_b	692.03 N	1000.6 (their $1.5 \cdot W_r$)
-------	-----------------	---------------------------------

T_b	38.06 N·m	55.03
-------	------------------	-------

F_t	692.03 N	420
-------	-----------------	-----

Report 9.0's $F_t = 420$ doesn't follow from its own stated formula $\mu \cdot F_v$ (which gives 692). That was one of the five unreproducible values in the earlier audit — your derived number is the defensible one.

PART D — A-arms

D.1 Physics

A wishbone is a **two-force member pair** — two legs, each carrying pure axial load, meeting at the ball joint. Report 9.0's treatment lumps the whole wishbone and applies three loads: its share of static weight, its share of longitudinal transfer, its share of lateral transfer.

D.2 Formulas

$$W = M_{\text{axle}} \cdot g / 4 \quad \text{quarter of the axle load}$$

$$F1 = M \cdot a_{\text{brake}} \cdot h / (4 \cdot L) \quad \text{longitudinal share}$$

$$F2 = M \cdot a_{\text{corner}} \cdot h / (4 \cdot t_{\text{axle}}) \quad \text{lateral share}$$

The $\frac{1}{4}$ is because each corner has two wishbones, and each takes half the corner's share.

D.3 MATLAB guidance

Write one local function taking axle mass and track width, call it four times. Front upper and front lower get identical inputs in this formulation, as do rear upper and rear lower — so two distinct calls produce four outputs.

Local functions go at the **end** of the file, after all the main code.

D.4 Expected values

Quantity	Value	Report 9.0
----------	-------	------------

Front $\frac{W}{2}$	279.59 N	279.58 ✓
---------------------	-----------------	----------

Rear $\frac{W}{2}$	333.54 N	333.54 ✓
--------------------	-----------------	----------

$\frac{F1}{2}$ (all four)	147.03 N	147.15
---------------------------	-----------------	--------

Front $\frac{F2}{2}$	232.41 N	232.7
----------------------	-----------------	-------

Rear $\frac{F2}{2}$	242.51 N	242.58
---------------------	-----------------	--------

All five within 0.15%. This is the component that reproduces Report 9.0 most cleanly.

PART E — Rockers

The most involved chain, and the one with a real finding in it.

E.1 Physics

Wheel moves up → pushrod pushes → rocker rotates → damper compresses.

Three lever stages, each a moment balance:

A-arm: the pushrod picks up *inboard* of the ball joint, so it sees more force than the wheel. Ratio $\frac{d_{BJ}}{d_{PR_arm}}$.

Pushrod angle: only the component of pickup motion along the pushrod axis does work.

Rocker: moment balance about the pivot, ratio $\frac{d_{PR}}{d_D}$.

E.2 Symbols

Symbol	Meaning	Your value
k	Spring rate	11.8 / 14.8 N/mm
MR	Motion ratio, wheel travel ÷ damper travel	1.429 / 1.592
K_w	Wheel rate	derived
δ_{bump}	Bump travel	25.4 mm
F_{bump}	Vertical wheel load at full bump	derived
d_{BJ}	A-arm pivot axis → ball joint	309.10 / 268.89 mm
$d_{\text{PR_arm}}$	A-arm pivot axis → pushrod pickup	254.88 / 211.07 mm
d_{PR}	Rocker pivot → pushrod line	131.40 / 82.86 mm
d_{D}	Rocker pivot → damper line	127.15 / 53.78 mm

E.3 Formulas

 $K_w = k \cdot MR^2$ ← see the note below

$F_{\text{bump}} = K_w \cdot \delta_{\text{bump}}$

$F_{\text{pushrod}} = F_{\text{bump}} \cdot (d_{\text{BJ}} / d_{\text{PR_arm}}) / \cos(\theta_{\text{pushrod}})$

$F_{\text{damper}} = F_{\text{pushrod}} \cdot (d_{\text{PR}} / d_{\text{D}})$

$R_{\text{pivot}} = -(F_{\text{pushrod}} \cdot \hat{0}_{\text{PR}} + F_{\text{damper}} \cdot \hat{0}_{\text{D}})$ vector sum

⚠ **The K_w convention is genuinely ambiguous on your car, and you should say so in the report.** Your LOTUS export defines MR as wheel ÷ damper travel. By virtual work that gives $K_w = k / MR^2$. But that yields **1.76 Hz** ride frequency — far below the 2.5–4.0 Hz FSAE band. Report 9.0's $K_w = k \cdot MR^2$ gives **3.58 Hz front, 4.47 Hz rear** — in range. The likely resolution: 11.8 and 14.8 N/mm are probably already *wheel* rates, mislabelled as spring rates. A real coilover spring at 11.8 N/mm (67 lb/in) would be extraordinarily soft for a race car. **For now use $K_w = k \cdot MR^2$** so your numbers are comparable to Report 9.0, and put the ride-frequency check in `report.m` as a flagged item. This is a genuine open question worth raising with whoever specced the springs.

E.4 ⚠ A real discrepancy to report

Chain your three lever stages together and you get an implied overall motion ratio. Compare against LOTUS:

Chain MR

LOTUS MR

Error

Front

1.392

1.429

-2.6% ✓

Rear

2.083

1.592

+30.8% ✗

The front closes nicely — three independent lever measurements reproducing LOTUS's full kinematic sweep to 2.6% is strong validation.

The rear does not. Something in the rear pushrod, rocker hardpoints, or the LOTUS rear MR is inconsistent. Don't chase it today — compute both, print both, flag it. It's a legitimate finding, and finding it is worth more than hiding it.

E.5 MATLAB guidance

Write front and rear as separate `%%` blocks — they use different A-arms (front lower, rear upper), so the geometry fields differ.

Before the pivot reaction: flip your damper direction. `geo.force_direct_rocker_damper_*` currently points chassis→rocker while the pushrod direction points rocker→A-arm. For a vector sum both must point consistently away from the rocker.

Add the cross-check as two lines: compute `F_bump × MR` and compare to `F_damper`, store the ratio.

E.6 Expected values ($\delta_{\text{bump}} = 25.4\text{ mm}$)

	Front	Rear
<code>Kw</code>	24.096 N/mm	37.510 N/mm
<code>F_bump</code>	612.04 N	952.76 N
<code>F_pushrod</code>	~825 N	~1288 N
<code>F_damper</code>	~852 N	~1984 N
<code>F_bump × MR</code> (check)	874.6 N	1516.8 N
Agreement	2.6% ✓	31% ✗

PART F — Hubs

F.1 Physics

The hub is the rotating part carrying the wheel studs and brake rotor, running on bearings around the upright. It sees three load contributors, which is how Report 9.0 structures it: braking torque, cornering force, bump force.

F.2 Formulas

EO
BB

$\text{decel} = u^2 / (2 \cdot S)$ from the braking event

$F_v = F_w + M \cdot \text{decel} \cdot h / (2 \cdot L)$

$f = \mu \cdot F_v$

$T_b = f \cdot r$ braking torque

$F_c = M \cdot u^2 \cdot h / (2 \cdot R \cdot T_f)$ cornering force

$F_{\text{bump}} = K_w \cdot \delta_{\text{bump}}$ from Part E

F.3 Two things to change from Report 9.0

Drop the gyroscopic term. Report 9.0 adds 1074.8 N as a vertical wheel load. Gyroscopic action produces a **moment**, not a wheel load, and at 11 m/s it's an order of magnitude smaller than that.

Use your derived loaded radius. Report 9.0's hub section uses 311 mm where its knuckle section uses 228.6 — a 36% swing in every torque. Yours is 234.80 throughout, consistently.

F.4 Expected values

Quantity	Value	Report 9.0
----------	-------	------------

decel	15.053 m/s² (1.534 g)	15.05 ✓
----------------	---	---------

F_v	935.26 N	1464.25
-------	-----------------	---------

f	673.38 N	878.55
-----	-----------------	--------

T_b	158.11 N·m	273.22
-------	-------------------	--------

F_c	1392.90 N	1392 ✓
-------	------------------	--------

T_b differs mostly from the tyre radius (311 vs 234.80).

Note the internal inconsistency you're now exposing: the hub section assumes 1.534 g deceleration while the knuckle section assumes 1.2 g, on the same car. Your program will make that visible for the first time. Worth a line in the report.

PART G — Rack and pinion

G.1 Physics

Three separate mechanisms twist the wheel about its steering axis, and the rack has to resist all of them.

Moment	Source
M_v	Vertical wheel load acting through scrub radius, modified by kingpin and steer angle
M_l	Lateral force acting through caster trail
M_t	Difference in tractive/braking force across the axle, through scrub radius

G.2 Symbols

Symbol	Meaning	Your value
<code>Rfl</code> , <code>Rfr</code>	Vertical reactions, front left / right derived	
<code>λ</code>	Kingpin inclination	3.609°
<code>δ</code>	Steer angle at full lock	30.08°
<code>v</code>	Caster angle	2.602°
<code>rs</code>	Scrub radius	63.79 mm
<code>Ls</code>	Steering arm	45.05 mm
<code>d_pinion</code>	Pinion pitch diameter	22.5 mm

G.3 Formulas



$$\text{Lateral transfer} = M \cdot 1.4g \cdot h / T_f$$
$$Rfl = Fw + \text{transfer_front}/2$$
$$Rfr = Fw - \text{transfer_front}/2$$
$$Mv = -(Rfl + Rfr) \cdot (rs \cdot \text{ind}(\lambda) \cdot \text{ind}(\delta)) + (Rfl - Rfr) \cdot (rs \cdot \text{ind}(v) \cdot \text{cosd}(\delta))$$
$$MI = -(Rfl + Rfr) \cdot (r \cdot \text{tand}(v)) \quad \text{with } Rfl = Rfr = 1.5 \cdot M \cdot g/4$$
$$Mt = (Rfl \cdot \mu - Rfr \cdot \mu) \cdot rs$$
$$M = |Mv| + |MI| + |Mt|$$
$$F_{\text{tierod}} = M / Ls$$
$$F_{\text{rack}} = F1 + F2 \quad \text{where } Fi = M / (0.97 \cdot Ls \cdot \text{cosd}(\text{angle}_i))$$
$$T_{\text{pinion}} = F_{\text{rack}} \cdot d_{\text{pinion}} / 2$$

Take `abs()` before summing. `Ml` computes negative but must be added as a magnitude. Without `abs()` you get roughly 3,100 instead of 43,800 — a factor of 14.

The **0.97 factor** is the projection of the steering axis onto vertical, from Report 9.0. The two tie rod angles (60° inner, 52.3° outer) differ because inner and outer wheels sit at different steer angles at full lock.

G.4 Expected values

Quantity	Value	Report 9.0
Rfl / Rfr	812.10 / 306.24 N	733.66 / 384.74
Mv	−983.01 N·mm	3124.04
Ml	−19626.90 N·mm	19094.34
Mt	23232.96 N·mm	16020.64
M total	43842.88 N·mm	38239.02
F_tierod	973.17 N	849.76
F_rack	3647.13 N	3184.61
T_pinion	41.03 N·m	35.83

T_pinion = 41 N·m is a design finding, not just a number. That's the torque at the steering wheel with a direct column. Driver comfort limit is 15–25 N·m. Your steering is roughly twice as heavy as it should be — worth stating explicitly, and it points at either the 45 mm steering arm being short or the 63.8 mm scrub radius being large.

PART H — report.m

H.1 What it should print

Four blocks:

- Geometry summary** — the values you already verified, so the report is self-contained.
- Force results by component** — every `res.*` field with its unit.
- Comparison against Report 9.0** — three columns: derived, published, % difference. This is the table that turns your divergences into documented findings.
- Flagged items** — the things a reader needs to know:

EU BB g a, b derived as 76.96/102.84 mm vs Report 9.0's 110.9/110.9 g Loaded radius 234.80 mm used throughout; Report 9.0 mixes 228.6 and 311 g Rear knuckle Fc, Fb derived from mu rather than 3x/1.5x multipliers g Rear rocker chain implies MR 2.083 vs LOTUS 1.592 — unresolved g Wheel rate convention ambiguous; k*MR^2 used, gives 3.58/4.47 Hz g Deceleration inconsistent between sections: 1.2 g knuckle, 1.534 g hub g Steering wheel torque 41 N.m, above the 15-25 N.m comfort band

H.2 MATLAB guidance

Use `fprintf` with fixed-width specifiers so columns align — `%-28s %12.2f %12.2f %8.1f%%\n` gives you a clean table. Remember `%%` for the literal percent and `\n` at the end of every line.

For the comparison table, build a **cell array** of `{name, derived, published}` triples and loop over it. Adding a row later is then one line.

Export with `array2table` and `writetable` to separate sheets — `Geometry`, `Forces`, `Comparison`.

PART I — Validation checks

Code these as `assert` calls so the script stops rather than producing plausible rubbish.

```
EO
EE

g Fv > Fw always          (transfer adds load in braking)
g f/Fv == mu exactly
g F1 + F2 == f            (force balance closes)
g Fnet > Fw               (cornering adds load)
g Ride frequency 2.5-4.0 Hz    flag, don't halt
g F_damper vs F_bump*MR within 10%  front passes, rear will flag
Every force magnitude < 6 kN
g T_pinion between 15 and 25 N.m  currently fails — report it
```

The last three are the valuable ones. They compare independent routes to the same quantity, so agreement validates your whole chain, and disagreement points at a real physical inconsistency rather than a typo.

PART J — Three-hour schedule

Time	Task	Checkpoint
0:00-0:15	Update <code>gen: bump</code> → 25.4, add <code>MR_F</code> / <code>MR_R</code> , fix the <code>c</code> comment. Write <code>main.m</code> .	<code>geo</code> prints without error
0:15-0:50	<code>frontknuckle.m</code> — three sections, unpack-in-metres block at the top	All 16 values match Part B.9
0:50-1:10	<code>rearknuckle.m</code> — same pattern, copy and adapt	6 values match Part C.3
1:10-1:30	<code>aArms.m</code> — one local function, four calls	5 values match Part D.4
1:30-2:05	<code>rockers.m</code> — front and rear blocks, fix damper direction, add the MR cross-check	Front check passes, rear flags
2:05-2:25	<code>hubs.m</code>	5 values match Part F.4
2:25-2:50	<code>rackpinion.m</code> — watch the <code>abs()</code> before summing	8 values match Part G.4
2:50-3:00	<code>report.m</code> — comparison table and flagged items	Runs end to end

If you run short, cut in this order: the export sheets (print to console instead), then the pivot reaction vector in `rockers.m`, then the flagged-items block. Never cut `frontknuckle.m` or the comparison table. **The one thing to be patient about is the first function.** Get `frontknuckle.m` exactly right — its unpack-in-metres block, its section structure, its `res` naming — and the other five are the same shape with

different formulas. Rushing it means repeating a unit bug five times.

When something doesn't match

Post the value and which function. Every number in this document came from your actual hardpoints and `gen`, so a mismatch is a specific bug, not a modelling question — and the intermediate values above will localise it to one line.